Dust Particles and Gas+Dust Self-Gravity in the AthenaK Shearing Box

Implementation Document

Phase 4a: the dust-module merge; Phase 4b (i): the shear-periodic deposit fold

`athenak-multigrid` tree, branch `dust-multigrid`

commits `449b8f09` (merge), `a7c0def3`, `774c6519`, `78320aee` (second merge)

September 3, 2026

Abstract

This document describes *how* Lagrangian dust particles are being brought into the `athenak-multigrid` tree — the shearing-box code with multigrid self-gravity on refined meshes documented in `multigrid_shear_implementation.pdf` — on the way to gas+dust self-gravity in a stratified, cooling, gravito-turbulent box (the configuration of Baehr, Zhu & Yang 2022, with the radial pressure gradient they omitted). It is the companion to `dust_selfgravity_validation.pdf`, which records what the resulting code does on test problems. Both documents are meant to grow phase by phase.

Phase 4a, recorded here, merges the sibling fork `athenak-dust` (particle-mesh dust with IMEX drag, validated in its own repository) into the multigrid tree. The merge itself is mechanical — eight keep-both conflicts and one renumbered output slot — but it exposed a real defect: the dust module's additive ghost-deposit exchange was written against the per-neighbor MPI messaging of July's upstream, while this tree carries the rank-packed messaging of upstream #775. Any multi-rank dust run deadlocked at cycle 0. The exchange now uses the aggregated-message path with the same idiom as the cell-centered exchange, and ranks 1, 2 and 4 agree to every printed digit. A second follow-up aligns the dust task graph with the tree's Prolongate-before-boundary-conditions order.

A **second merge** the same day caught the tree up with the dust fork's own latest work (seven commits, to `1f3e1ed9`): the fork had independently made the identical rank-packed rewrite of the deposit exchange, so that conflict resolves to their version; and it now offers two *predictor–corrector* coupling modes (`coupling = pc2` and the adaptive `hybrid`) that run under plain `rk2`, second order in the resolved drag regime with a stiff backward-Euler fallback. Dust can therefore run under the same integrator as every gravity validation before it; the `imex2+` path remains for stiff cases.

Phase 4b (i) closes the first of the gaps the survey named: ghost deposits of the MeshBlocks on the shear-periodic x_1 faces are now folded across the faces with a conservative azimuthal remap — gathered into global y–z planes, shifted by the shear offset with the Phase-1 remap-flux kernels, and *added* into the active edge strips of the blocks across the face — while the unsheared plain-periodic x_1 contributions of those blocks are suppressed. The fold is the adjoint of the copy-exchange ghost fill, so its shift signs are the reverse of the multigrid fill's. A unit-test problem generator certifies it exactly at integer shifts and at second/third order otherwise, and a structured 3D run shows the old unsheared exchange was a percent-level error.

The document also records, for the phases that follow, the integration map derived from the code survey: where the gravity solve is called relative to the dust task graph, the three places the solver reads density, the ghost-width contract of the potential, and the design chosen for the particle force (grid gradient, exchanged and shear-remapped like the module's provisional gas

velocity, gathered with the deposit kernel), together with the known gaps that Phases 4b–4d must close.

Contents

1 Scope, sources, and how to read this

1.1 What this document is

This is an implementation reference for the dust track. For each phase it answers: what the code relies on, where that is established and enforced, which data structure carries the new state, which function does the work, and from which call site it is reached. Results live in the companion documents:

`validation/dust_selfgravity_-validation.pdf`	results, tests, figures (this track)
`validation/multigrid_shear_-implementation.pdf`	the gravity/refinement machinery being extended
`validation/multigrid_selfgravity_-validation.pdf`	its validation record (Phases 0–3)
`../athenak_dust_tests/`	the dust module's own validation notebooks and inputs

1.2 The program

The target science is the interaction of gravitational instability of the gas with dust concentration and collapse in a shearing box: Baehr, Zhu & Yang (2022, ApJ 933, 100) in 3D stratified, cooling, gravito-turbulent boxes with superparticles at $\mathrm{St} = 0.1$–10, plus the radial pressure gradient (and hence the streaming instability) that their setup left out, and eventually adaptive zoom on the dust clumps. The phases:

Phase	Status	Content
4a	**done**	merge `athenak-dust` into the multigrid tree; establish that both the dust module and the gravity machinery are unchanged by the merge
4b	(i) **done**	(i) shear-periodic *additive* deposit remap (§5); (ii) ideal-gas energy update under drag back-reaction; (iii) particle timestep on the shear-subtracted velocity; adopt the PC2 coupling (§4.5) as the production path
4c	planned	dust self-gravity: mass deposit into the Poisson source, three density reads, force gather to particles; dust-only and gas+dust tests
4d	planned	the science configuration: SC14 box + pressure gradient + dust; particle restart; clump diagnostics
5	deferred	particles on refined meshes (deposit exchange across levels, particle redistribution on remesh)

1.3 Commit provenance

The merged fork is `github.com/athenak-dust/athenak`, branch `dust`. At the first merge it stood at `d6de1296`, thirteen commits on top of upstream AthenaK as of July 8, 2026 (commit `5f199310`, the merge base with this tree). The one that matters is `e785735e` "Add IMEX dust-particle coupling to hydrodynamics" (36 files, +3864 lines); the rest are the matrix-free coupled drag solvers, GPU migration fixes, a snapshot-seeded clump benchmark, and diagnostics. The local clone turned out to be two weeks stale; the second merge (§4.5) brought in the seven commits through `1f3e1ed9` (September 2). The remote is registered locally as `dust` (`git remote add dust ../athenak-dust`),

Commit	Phase	Content
`449b8f09`	4a	Merge of `dust/dust` (`d6de1296`) into `multigrid` (`b4d4c547`); eight conflicts resolved keep-both; output slot `grav_phi` 153 → 154
`a7c0def3`	4a	Dust task graph: `Prolongate` before `ApplyPhysicalBCs`
`774c6519`	4a	`bvals_dep.cpp`: additive deposit exchange on the rank-packed MPI vars path (multi-rank deadlock fix)
`78320aee`	4a	Second merge, `dust/dust` at `1f3e1ed9`: the fork's own rank-packed deposit exchange (taken), PC2/hybrid coupling, PM optimizations; three conflicts (§4.5)
(next)	4b (i)	`MeshBoundaryValuesDep::FoldShearDeposit`, the x_1-buffer skip, the dust hooks, the `dust_deposit_shear` test generator, the 3D shear-periodic NSH input, the NSH mass modulation

Table 1: Implementation commits covered by this document.

so later fixes in that fork come in with a plain `git merge dust/dust` — which is exactly what the second merge was.

1.4 Files touched

File	Role
`src/dust/dust.{cpp,hpp}`	`DustGasDrag` module: parameters, arrays, boundary objects, guards, the shared PM kernel `dust::PMWeights`
`src/dust/dust_tasks.cpp`	the combined hydro+particle task graph (`AssembleDustGasDragTasks`) and the boundary tasks
`src/dust/dust_pm.cpp`	deposit, per-cell implicit solve, gather/kick, back-reaction
`src/dust/dust_push.cpp`	explicit particle push (Coriolis, tide, drift, drag history)
`src/dust/dust_newdt.cpp`	particle transport timestep, stopping-time bookkeeping, the γ-switch
`src/dust/dust_solver.cpp`	optional matrix-free coupled drag solvers
`src/bvals/bvals_dep.cpp`	`MeshBoundaryValuesDep`: additive ghost-deposit exchange (rewired in 4a)
`src/bvals/bvals_part.cpp`, `bvals.{cpp,hpp}`	shear-periodic particle routing; deposit-exchange class declaration
`src/driver/driver.{cpp,hpp}`	`SetImEx2PlusCoefficients` (the γ-switch entry point)
`src/mesh/meshblock_pack.{cpp,hpp}`	`pdust` registration; hydro/particle task lists ceded to the dust module when a `<dust>` block exists
`src/mesh/mesh.{cpp,hpp}`, `build_tree.cpp`	`<time>/dtmax` hard cap
`src/athena.hpp`	widened `ParticlesIndex` (17 real, 3 integer fields)
`src/outputs/*`	`dust_d` derived output; particle velocities in `pvtk`; slot renumbering
`src/pgen/tests/dust_*.cpp`, `streaming_linear.cpp`	the four built-in dust problem generators
`inputs/dust/*.athinput`	their reference inputs

2 The target problem and the design it implies

2.1 What is being built

The reference implementation for particle self-gravity in a shearing box is the Athena-C problem generator `parsg_strat3d.c` in `athena_parsg/src/prob/` (Rixin Li's fork). Its method, with `PLUS_GAS_SELF_GRAVITY`: deposit the dust mass onto the grid with the TSC kernel, *add the gas density into the same buffer*, solve *one* Poisson equation, difference Φ on the grid, and interpolate the resulting acceleration to the particles with the *same* TSC kernel used for the deposit. The gas feels the total potential as a body force. Baehr et al. do the same in Pencil (their eq. 15, $\rho = \rho_g + \rho_d$ in a periodic FFT). Two details of the Athena-C version are not to be copied: its deposit omits the cell-volume division and hides it in the particle mass, and its drag feedback must subtract the gravitational kick because gravity is folded into the same implicit particle update as drag.

2.2 The integration map

The survey of this tree fixed where each piece attaches. These are facts about the code as merged; the items marked *planned* are the Phase 4b/4c designs.

Where gravity is solved. Gravity has no task-list entry. The driver calls `Gravity::Solve(stage)` once per explicit stage, *between* the `before_stagen` and `stagen` task lists (`driver/driver.cpp`, lines 563–569). Anything the solver must see — the dust mass deposit and its ghost exchange — has to complete inside `before_stagen` (*planned*: a deposit task there, on the stage-start particle positions).

The three density reads. The solver reads gas density in three places, and all three must learn about dust (*planned*, Phase 4c):

- `gravity/mg_gravity.cpp:353`, `LoadSource(u0, IDN, ...)` into the multigrid source: loads active zones *plus one ghost layer*, so the dust deposit must already be additively ghost-exchanged one layer deep;
- `multigrid/multigrid_slab.cpp:263`, the slab-plane gather that builds the open-z Dirichlet planes (active zones, reduced across ranks) — omitting dust here would make the planes inconsistent with the interior source;
- `gravity/fft_gravity.cpp:218`, the FFT oracle's density gather.

The potential's ghost contract. `phi` is allocated with the hydro ghost width but only *one* ghost layer is ever filled (`mg_nghost = 1`; larger values are fatal with shear). The gas source term (`SourceTerms::SelfGravity`, `srcterms/srcterms.cpp:298`) needs exactly that: a centered $-\rho\nabla\Phi$ for momentum and the flux-based energy term of Mullen, Hanawa & Gammie (2020). A TSC gather at a particle in the last active cell reaches one ghost cell of the *acceleration*, i.e. two of the potential. *Planned*: compute $\mathbf{g} = -\nabla\Phi$ on active cells, then copy-exchange and shear-remap the three-component field exactly as the module already does for its provisional gas velocity u^* (`MeshBoundaryValuesCC` plus a `ShearingBoxCC`), and gather with `dust::PMWeights`. Using the deposit kernel for the force gather — not the derivative of the kernel — is the Hockney–Eastwood choice that suppresses self-force, and it is what the Athena-C reference does.

Where the particle force goes. `DustGasDrag::ExplicitPush` (`dust/dust_push.cpp:65`) already assembles the Coriolis and vertical-tide accelerations per particle; the gathered $\mathbf{g}$ joins them there. The back-reaction deposits only the drag kick $\Delta v_j = c_j(\tilde{u}_j^* - v_j)$, so gravity is never double-counted — there is no analogue of the Athena-C `feedback_corrector` subtraction to write.

The gas force. Nothing to add: the existing source term applies whatever potential the solver produced.

Stage structure. Under `coupling = imex`, `imex2+` has three explicit stages of which the third is dormant for explicit physics ($\beta_3 = 0$); the gravity solve should be skipped there or a third of the solver cost is wasted (*planned*). Under the PC2/hybrid couplings of §4.5 the integrator is `rk2` and the gravity solve runs exactly as it does for gas alone.

3 The dust module as inherited

This section is the working summary of `athenak-dust`, with the facts the later phases build on. The design is Yang & Johansen (2016) particle-mesh coupling (one shared kernel for scatter and gather, exact momentum-conserving back-reaction) with the Benítez-Llambay, Krapp & Pessah (2019) closed-form per-cell implicit drag inside the Krapp et al. (2024) IMEX(4,3,2) integrator (`imex2+`), generalized to per-particle stopping times.

3.1 Data layout

Particles are two flat device arrays indexed `(field, particle)`. The field enumeration (`athena.hpp:76`) is

```
enum ParticlesIndex {PGID=0, PTAG=1, PSP=2,
                     IPX=0, IPVX=1, IPY=2, IPVY=3, IPZ=4, IPVZ=5,
                     IPX1=6, IPVX1=7, IPY1=8, IPVY1=9, IPZ1=10, IPVZ1=11,
                     IPRX=12, IPRY=13, IPRZ=14, IPTS=15, IPM=16};
```

so a dust particle carries position, velocity (all three components even in 2D r–z), the low-storage stage-1 registers, the recorded drag rate R_j, its stopping time and its **mass**. Integer data: owning MeshBlock, global tag, species. `nrdata = 17`, `nidata = 3`.

3.2 The particle-mesh kernel

`dust::PMWeights` (`dust/dust.hpp:78`) returns the three weights of a NGP / CIC / TSC stencil in one direction; every deposit, gather, matrix-free operator and kick uses it, which is what makes scatter and gather weights identical by construction. `<dust>/deposit` selects the kernel (default `tsc`); the stencil is always three wide (NGP zeroes the wings). Particle-mesh assignment needs `nghost >= 2`.

3.3 The stage algorithm

Per explicit stage, on the conserved gas state (primitives are stale mid-stage), with $a\,\Delta t = a_{\rm impl}\Delta t$ the implicit weight of the tableau:

1. `DepositDrag`: scatter $Q = \sum_j \mu_j W_{jk}$ and $\mathbf{P} = \sum_j \mu_j W_{jk}\mathbf{v}_j$ with $c_j = a\Delta t/(t_{s,j} + a\Delta t)$, $\mu_j = m_j c_j / V$ (a *drag-weighted* density, not a mass density); additive halo exchange.

2. `GasImplicitSolve`: per cell, $\mathbf{u}^* = (\rho\mathbf{u} + \mathbf{P})/(\rho + Q)$, the closed-form momentum-conserving backward-Euler pair. With `back_reaction = false` the deposits are excluded so test particles are kicked toward the unmodified gas.
3. Copy exchange of $\mathbf{u}^*$ ghosts (plus shear remap in a 3D shearing box).
4. `GatherKickPMBR`: gather $\tilde{\mathbf{u}}_j^*$, kick $\Delta\mathbf{v}_j = c_j(\tilde{\mathbf{u}}_j^* - \mathbf{v}_j)$, record $R_j = \Delta\mathbf{v}_j/(a\Delta t)$, and deposit $-m_j W_{jk}\Delta\mathbf{v}_j/V$ with the *same* weights and the same $\Delta\mathbf{v}$ — exact momentum conservation to round-off.
5. Additive halo exchange of that deposit; `ApplyPMBR` adds it to the gas momentum and rescales it in place into the gas drag rate R_g.
6. Next stage, the history terms $\tilde{a}_{22}\Delta t\, R$ enter on both sides (`AddDragHistoryGas`; the particle side inside `ExplicitPush`).

There is *no* drag timestep constraint for any t_s or dust-to-gas ratio; only particle transport limits Δt (§3.6).

3.4 The task graph

When a `<dust>` block exists, `MeshBlockPack::AddPhysics` skips both `Hydro::AssembleHydroTasks` and `Particles::AssembleTasks` and hands the whole graph to `DustGasDrag::AssembleDustGasDragTasks` (`mesh/meshblock_pack.cpp:265-271`, `dust/dust_tasks.cpp:37-108`), the same pattern as `IonNeutral`. The consequence for later phases:

Ownership. The dust module owns the entire per-stage task graph. Any task the gravity work needs (deposit, exchange, force gather) is inserted into `AssembleDustGasDragTasks`, and the hydro chain inside it must track changes made to `Hydro::AssembleHydroTasks` in this tree (the 4a follow-up `a7c0def3` is the first instance).

The `stagen` sequence: `FirstTwoImpRK` (replaces `CopyCons`) → gas fluxes / update / source terms → `AddDragHistoryGas`; in parallel `ExplicitPush` → particle migration (`NewGID`, counts, sends, receives — *inside every stage*, not once per cycle) → the deposit chain of §3.3 → the gas tail (orbital advection, restrict, exchange, shear exchange, `Prolongate`, `ApplyPhysicalBCs`, `ConToPrim`, timesteps). Every dust task returns early on the dormant third stage.

3.5 Shearing-box conventions

Particle velocities are *shear-subtracted*: the explicit push applies the FARGO-form Coriolis terms $f_x = 2\Omega_0 v_y$, $f_y = -(2-q)\Omega_0 v_x$ and, with `<shearing_box>/stratified`, $f_z = -\Omega_0^2 z$ (`dust/dust_push.cpp:98-100`); positions drift with the *full* azimuthal velocity, $\dot{y} = v_y - q\Omega_0 x$ (`:113-114`). This matches the gas with `orbital_advection = true` (the tree's default and the SC14 setting). Crossing a radial face is therefore a *position-only* transform: wrap x, shift y by $\mp y_{\rm sh}$, $y_{\rm sh} = q\Omega_0 L_x t \bmod L_y$ (`bvals/bvals_part.cpp:76,134`), with the RK position registers shifted by the same amounts so the low-storage combination stays in one periodic image. The destination MeshBlock is found from a precomputed $(y, z) \to$ (gid, rank) map of the face blocks, not from the x_1 neighbor.

The radial pressure gradient is a gas-only constant acceleration through the stock `<hydro_srcterms>/const_accel` machinery ($2\Omega_0\eta v_K$ along x); particles never feel it.

3.6 The particle timestep

`DustGasDrag::NewTimeStep` (`dust/dust_newdt.cpp:95`) takes $\min_p \min_i \Delta x_i/|v_{\mathrm{transport},i}|$ times `<dust>/dt_cfl` (default 0.5), where in a 3D shearing box $v_{\mathrm{transport},y} = v_y - q\Omega_0 x$ (`:120`). In a wide box this is the same penalty the gas pays without orbital advection ($\sim 5\times$ at SC14 size). Phase 4b relaxes it (§6).

3.7 Guards and parameters

Fatal conditions in `DustGasDrag::DustGasDrag` (`dust/dust.cpp`): MHD present (`:45`); 1D mesh (`:50`); an integrator other than `imex2+` (`:63`); `multilevel` (`:69`); boundaries neither strictly periodic nor 3D with shear-periodic x_1 (`:78`); `nghost < 2` (`:93`); back-reaction with a non-isothermal EOS (`:170`); a coupled solver other than `local` with shear (`:227`). A *warning* at `:84` states that ghost deposits are exchanged with the plain-periodic pattern under shear, exact only for azimuthally uniform states. The same `multilevel` guard is repeated in the `MeshBoundaryValuesDep` constructor and in the shear-periodic particle constructor. The parameter interface is reproduced in §7.

4 Phase 4a: the merge

4.1 Route

The two forks share upstream history through `5f199310` (July 8, 2026); the dust fork adds 13 commits, this tree about 140 including an upstream merge on August 10 that brought in #775 (rank-packed boundary messaging) and #612 (`imex2+`). A merge was preferred over cherry-picks: it keeps the dust fork's history and authorship, resolves conflicts once, and makes later fixes from that fork a one-line pull. The whole sequence was first rehearsed in a scratch clone (conflicts, follow-ups, both builds, the full acceptance battery) and then repeated in the working tree with the rehearsed resolutions copied in.

4.2 Conflicts

Eighteen files were touched by both forks; eight conflicted, all of the same kind — both sides appended adjacent code at the same anchor — and all were resolved by keeping both sides:

File	this tree had added	the dust fork had added
`driver/driver.cpp`	the super-time-stepping controller functions	`SetImEx2PlusCoefficients` (the constructor's inline `imex2+` tableau moved into a function the γ-switch can call)
`mesh/mesh.cpp`, `mesh.hpp`	STS timestep cap and members	`dtmax` hard cap (default ∞, inert)
`mesh/meshblock_pack.cpp`, `.hpp`	physics (9) gravity, `pgrav`	physics (10) dust, `pdust`
`outputs/outputs.hpp`, `basetype_output.cpp`	`grav_phi` at slot 153 and its guard	`dust_d` at slot 153 and its guard
`parameter_input.cpp`	`gravity` in the block-name list	`dust`

The output slot. Output variable names live in one static array, `var_choice`, and the index of the matching name (`ivar`) is used only by the sanity guards that test numeric ranges ("151–153 are particle outputs, so a `<particles>` block must exist"). Each fork had appended one name at

position 153 and written a guard for it. Resolution: `dust_d` keeps 153, `grav_phi` becomes 154, `NOUTPUT_CHOICES` 154 → 155, the gravity guard tests 154. Everything downstream selects by name string, so the number appears nowhere else. The 31 auto-merged files were verified identical to the rehearsal.

4.3 Follow-up 1: task order

Upstream reordered the hydro chain to `Prolongate` → `ApplyPhysicalBCs` → `ConToPrim` (so physical boundaries act on prolongated ghosts); the dust graph still had the July order. Commit `a7c0def3` aligns `dust/dust_tasks.cpp:96-98`. Inert on uniform meshes (dust is fatal on refined ones), but the invariant of §3.4 says keep them in lockstep.

4.4 Follow-up 2: the deposit exchange on the rank-packed path

Symptom. The first multi-rank test of the merged tree (the damping problem with drifting particles, `v0 = 0.5`, 4 ranks) hung at cycle 0 with all four processes spinning; the 1-rank run of the same binary reproduced the reference number exactly, and multigrid tests at 4 ranks were bit-identical to serial.

Mechanism. `MeshBoundaryValuesDep` (`bvals/bvals_dep.cpp`) was written as a copy of July's `MeshBoundaryValuesCC`: per-neighbor `MPI_Isend` on `sendbuf[n].vars_req[m]` with `CreateBvals_MPI_Tag(lid, dn)` tags, and per-neighbor `MPI_Test` on `recvbuf[n].vars_req[m]`, relying on the base class `InitRecv` to post the matching per-neighbor receives. In this tree `MeshBoundaryValues::InitRecv` (`bvals/bvals_tasks.cpp:32`) is *rank-packed* since upstream #775: it builds, once per mesh sequence, a metadata table of all (block, neighbor) payloads per peer rank (`BuildRankPackedVarMetadata`, `bvals/bvals.cpp:134`, which also does a one-shot header exchange so receivers know the pack order), and posts *one* `MPI_Irecv` per peer rank with tag 1 into `rank_recvbuf_vars_`. `ClearRecv`/`ClearSend` wait on those aggregated requests. So the deposit's per-neighbor sends had no receiver, its tests polled requests that were never posted, and the stage never completed.

Fix. Commit `774c6519` re-expresses the two transport steps in the same idiom as the cell-centered exchange (`PackAndSendCC` and `RecvAndUnpackCC` of `MeshBoundaryValuesCC`):

- `PackAndSendDeposit` (`:122`): on-rank neighbors are still written straight into the destination's `recvbuf[dn].vars`; off-rank payloads go into `rank_sendbuf_vars_` at `send_agg_offset_(m*nnghbr+n)`; after one fence, one `MPI_Isend` per message in `send_var_msgs_` (tag 1, `send_var_reqs_`);
- `RecvAndSumDeposit` (`:228`): `MPI_Test` over `recv_var_reqs_`; in the (deliberately scalar) loop over neighbors the addend is `aggrbuf(base + bi)` when `recv_agg_offset_(m*nnghbr+n) >= 0` and `rbuf[n].vars(m, bi)` otherwise.

The metadata builder sizes each entry from the buffer index ranges the derived class defines (`GetVarDataSize` → `isame_ndat`), so the reversed data flow of the additive exchange (pack ghosts, sum into active strips) needs no change to the base class. The serial path is unchanged (bit-identical damping ladder); ranks 1, 2 and 4 now agree to every printed digit.

Messaging. Every new boundary-exchange class in this tree must use the rank-packed vars path (`send_var_msgs_` / `recv_var_reqs_` / `*_agg_offset_`); the per-neighbor `vars_req` arrays are dead for vars traffic and only the flux-correction path still posts per-neighbor requests.

4.5 The second merge: catching up with the dust fork

What came in. A question about a colleague's commit revealed that the local dust clone was two weeks behind. `origin/dust` at `1f3e1ed9` adds, oldest first: a CPU particle-mesh optimization (plain addition instead of atomics under `Kokkos::Serial`, weights evaluated in cell coordinates); the NSH generator without dust; *their own* import of upstream #775 (`0d7688b6`), extended to the additive deposit exchange — the same rewrite as `774c6519`, validated by them at 2, 4 and 8 ranks and on CUDA; adaptive PC2 / split-BE coupling (`4b44ba09`); and direct PC2 (`7fc41cd8`).

Conflicts. Three files. `bvals/bvals_dep.cpp`: both sides rewrote the same functions the same way; the fork's version was taken (it additionally rebuilds the rank-packed metadata inside `PackAndSendDeposit` when the variable count or mesh sequence changed, instead of relying on `InitRecv` having run). `bvals/bvals_fc.cpp`: ours kept — the early return on inactive face-centered faces is upstream #739's $\nabla{\cdot}B$ fix, which the fork's #775 import predates. `mesh/mesh_refinement.cpp`: ours kept — the fork calls `InitBoundaryValuesAndPrimitives` at the ancestor's position, while the Phase-0c code here calls it later, after the shear-state rebuild; keeping both would call it twice. The base boundary-value files auto-merged with *no net change*: the fork's #775 import is textually the upstream copy this tree already had.

The PC2 and hybrid couplings. `<dust>/coupling` now selects `imex` (the default, §3.3, requires `imex2+`), `hybrid`, or `pc2` (both require `rk2`; `gamma_switch` is ignored with a warning). From the fork's own description: the *PC2* branch advances the particles once per cycle without a coupled gas solve — a fresh old-state drag-feedback predictor is deposited during RK stage 1, the midpoint gas velocity is built after the completed stage-1 boundary tail, the particles take an implicit-midpoint drag update at the predicted midpoint position, the actual drag impulse is deposited at that midpoint, and the final gas momentum is corrected by removing the tableau-weighted predictor. Particles migrate once, after the update. The *split-BE* branch first completes a dust-free hydro RK2 step and then performs one full-Δt coupled backward-Euler drag solve (first order, L-stable). `hybrid` switches between them globally with hysteresis on two stiffness measures,

$$\zeta = \max_p \frac{\Delta t}{t_{s,p}}, \qquad \chi = \max \frac{\Delta t\, \rho_d}{\rho_g\, t_s}, \tag{1}$$

entering split-BE when either exceeds `hybrid_enter_{zeta,chi}` (0.5) and returning to PC2 only when both fall below `hybrid_exit_{zeta,chi}` (0.25); `hybrid_force_mode = pc2 | split_be` pins a branch, and `coupling = pc2` is that pin made permanent. The choice is reduced across ranks so every rank takes the same branch, and a `DUST_HYBRID_SUMMARY` line reports the cycle counts and the last ζ, χ. In the task graph (`dust/dust_tasks.cpp`) this appears as per-stage gates on every exchange and two migration chains; the deposited $Q, \mathbf{P}$ field gains a fifth feedback-rate channel; a second timestep task follows the split-BE migration.

Consequence for the program. The science targets (St = 0.1–10 at $\Delta t \sim 10^{-2}\,\Omega_0^{-1}$) sit in the resolved regime $\zeta \ll 1$, so `coupling = pc2` under `rk2` is the natural production setting: second order, about twice the speed of the IMEX dc1 path by the fork's measurement, no `imex2+` CFL penalty, and the same integrator as all gravity validation. Note that `hybrid` with the three-species NSH test ($t_{s,\min} = 0.01$) already trips $\zeta = 0.55$ and runs split-BE throughout; the validation document records the first-order signature that produces. The IMEX path stays as the reference for stiff configurations.

4.6 Build notes

`imex2+` needed nothing: it is upstream #612 and was already in the tree; the dust fork only refactored its coefficients into `SetImEx2PlusCoefficients`. Dust sources are part of every build (`src/CMakeLists.txt`); the four dust problem generators are built-in (`-D PROBLEM=built_in_pgens`, selected by `<problem>/pgen_name`). The rehearsal clone was built without network by linking the tree's `kokkos` checkout and pointing `FETCHCONTENT_SOURCE_DIR_KOKKOS-FFT` at the cached `build/_deps/kokkos-fft-src`.

5 Phase 4b (i): the shear-periodic deposit fold

5.1 The problem

A particle near a MeshBlock boundary deposits part of its weight into ghost cells of its own block; the additive exchange (§4.4) adds those ghost deposits into the active cells of the neighbor that owns the location. Across a shear-periodic x_1 face the owner's cells sit at a *different* y: the tree wraps `shear_periodic` exactly like `periodic` (`mesh/meshblock_tree.cpp`), so the x_1 buffers of the face blocks carry the deposits to the unshifted y — the "exact only for azimuthally uniform states" caveat the module used to print. The copy exchanges of hydro and of u^* fix this by *overwriting* the x_1 ghosts with a remapped copy after the plain exchange; an additive exchange cannot overwrite, so the unsheared contribution has to be suppressed and replaced.

5.2 The sign

The shearing-box identification used throughout the tree is $u(x, y) = u(x + L_x, y - q\Omega_0 t)$: the copy exchange fills the inner-x_1 ghosts with the outer boundary's content shifted by $+y_{\rm sh}$ ($y_{\rm sh} = q\Omega_0 L_x t$), and the outer ghosts by $-y_{\rm sh}$ (`multigrid/multigrid_shear.cpp:144-146`, `gravity/fft_gravity.cpp:414-416`). A *deposit* in an inner ghost cell at y is a contribution to the outer boundary at $y - y_{\rm sh}$; the deposit map is the adjoint (transpose) of the ghost-fill map, so its signs are reversed:

Adjoint signs. Inner-ghost deposits are shifted by $-y_{\rm sh}$ and added to the *outer* face blocks; outer-ghost deposits by $+y_{\rm sh}$ into the *inner* face blocks. Copying the multigrid fill's per-face signs verbatim gives the wrong answer.

5.3 The fold

`MeshBoundaryValuesDep::FoldShearDeposit(a, yshear, rcon)` (`bvals/bvals_dep.cpp`) reuses the Phase-1 global-plane machinery of `Multigrid::FillShearGhostPack` with three changes: the gather reads *ghost* slabs and is additive, the signs are the adjoint ones, and the scatter *adds* into active cells.

1. **Gather.** Two planes `shear_plane_(face, v*ng+d, gk, gj)` of the global y–z extent (uniform grid: `mesh_indcs.nx2`×`nx3`), zeroed, then every face block adds its whole x_1 ghost slab — all k, j *including the y/z corner ghosts* — at the periodically wrapped global index (face 0 ← inner ghosts `is-1-d`, face 1 ← outer ghosts `ie+1+d`). The corner rows overlap the active rows of the y/z neighbors' slabs, so the gather is an atomic *sum*: unlike the multigrid fill, several blocks legitimately write the same plane cell. Rows beyond a non-periodic z face are dropped (the vertical-boundary policy of §6 item 7; the dust guards currently require periodic z).

2. **MPI seam.** `Kokkos::fence` and one `MPI_Allreduce(SUM)` of each plane, reached by every rank in lockstep (the fold is a task; all ranks run the same graph, and every send a rank waits for is posted before the collective).

3. **Remap.** Per (face, variable, depth, z-row): the integer part of the shift is folded into a periodically wrapped scratch load, the fractional part `eps = fmod(ysh, dy)/dy` goes through `DC/PLM/PPMX_RemapFlx` (`shearing_box/remap_fluxes.hpp`) and the conservative update $q_j \leftarrow q_j - (F_{j+1} - F_j)$; $dy = L_y/\mathtt{gny}$ is the global rank-consistent cell width (the `ConsistentDx2` lesson). Face 0 uses $-y_{\rm sh}$, face 1 $+y_{\rm sh}$.

4. **Scatter-add.** Face 0 into the outer face blocks' `ie-d`, face 1 into the inner blocks' `is+d`, active k, j only.

The remap preserves the row sum, so the total deposited mass is conserved to round-off; the summation order in the gather and the reduction can differ with the rank count at round-off level only.

5.4 Suppressing the unsheared contribution

`RecvAndSumDeposit` skips, on a block whose inner (outer) x_1 face is shear-periodic, every receive buffer whose direction has $o_{x1} < 0$ (> 0): faces, x_1x_2 and x_3x_1 edges, and corners — all of which carry x_1-ghost content of the neighbor across the face. The direction of buffer index n comes from a 56-entry table built in the constructor from the slot layout documented in `mesh/nghbr_index.hpp` (faces 0–7, x_1x_2 edges 16–23, x_3x_1 edges 32–39, corners 48–55) and cross-checked against `NeighborIndex` at construction.

A pitfall worth recording. The first version built that table by calling `NeighborIndex` over all direction and sub-index arguments. For edges the function uses only the first sub-index, and the second landed on *other* slots — the x_3x_1 edges of one sign overwrote those of the other. The plain pass then still delivered the unsheared x_3x_1-corner deposits, which the unit test caught as an off-by-one in the multiplicity check (§5.6) and as a 10^{-4} drift of the 3D NSH equilibrium.

5.5 Hooks and parameters

Two tasks, `DustGasDrag::FoldDepQP` and `FoldPMBR` (`dust/dust_tasks.cpp`), follow `RecvDepQP` and `RecvPMBR` with the same per-coupling stage gates; the implicit solve now depends on the fold. `CompleteAddExchange` (coupled solvers) calls the fold as well. The offset is `DustGasDrag::ShearOffset()` $= q\Omega_0 L_x\, t$ at `pmesh->time` on every stage, the convention of the u^* remap. `<dust>/shear_remap` selects the order (`plm` default, as for u^*; `ppmx` available), `<dust>/shear_fold = false` is a diagnostic that skips the fold (the face blocks' x_1 deposits are then dropped, not unsheared). The warning the module printed under shear is replaced by an information line.

5.6 The test generator

`pgen_name = dust_deposit_shear` (`pgen/tests/dust_deposit_shear.cpp`, `inputs/dust/dust_deposit_shear.athinput`) exercises the exchange without particles: the x_1 ghost slabs of the face blocks are filled with known data, every other cell is zero, and the exchange runs exactly as the module runs it. Ghost deposits are *partial* sums (a block's corner ghosts overlap its neighbors' active rows), which the first version of the test got wrong by filling every ghost cell with the full function;

the corrected test carries three fields: a smooth $g(x, y, z)$ periodic in y, z on the slab rows inside the block's own y/z range (one source per location, so the strips must equal g at the shifted location up to the remap error), a constant on the same rows (exact for any shift), and a constant on the *whole* slab whose received value must equal the analytic overlap multiplicity $(1 + n_y)(1 + n_z)$ of the source row (exact at integer shifts). The generator also reports the largest value leaked into any active cell outside the receiving strips (must be exactly zero) and the relative mass change (round-off). `problem/fold = false` runs the plain pass alone.

5.7 A structured, rank-independent initial condition

The NSH generator's random placement hashes each particle's tag to a block index *within the local pack*, so the realization depends on the rank count (as does `dust_damping`'s, seeded by the pack's first GID) — fine for physics runs, useless for serial-vs-MPI comparisons; a divergence that this produced was first mistaken for a bug in the fold (the no-back-reaction control agreed to 10^{-13} because test particles never feed positions back). `dust_nsh` therefore gained `<problem>/mass_mod_amp`, `mass_mod_phase`: on the lattice, particle masses are multiplied by $1 + A\sin(2\pi(y - y_{\min})/L_y + \phi)$, a deterministic azimuthally structured dust distribution (with `check_equilibrium = false`). `inputs/dust/dust_nsh_shear3d.athinput` is the tracked 3D shear-periodic NSH input with all Phase-4b keys present for command-line overrides.

6 What the next phases must add

The survey established these gaps; they are listed in the order they bite.

1. **Shear-periodic additive deposit remap (4b). Done**, §5. The Phase-4c gravity deposit will be a third additive field and inherits the fold through `MeshBoundaryValuesDep`. The alternative design (image particles deposited directly in the neighbor's frame) stays in reserve; the field remap is exact for constants and second/third order otherwise, and it costs one small collective per fold.

2. **Ideal-gas back-reaction (4b).** `ApplyPMBR` updates momentum only, so back-reaction requires an isothermal EOS, while gravito-turbulence needs cooling. Add $\Delta E = \Delta(|\mathbf{M}|^2/2\rho)$ (internal energy fixed under the drag kick), with frictional heating as a flagged option.

3. **Particle timestep under shear (4b).** Limit on v_y, not $v_y - q\Omega_0 x$, with a guard that the per-stage azimuthal jump stays below a MeshBlock width (migration is position-based and finds the destination by (y, z) lookup, so multi-cell jumps are legal). Otherwise every dust run in a wide box pays the non-FARGO factor.

4. **Dust self-gravity (4c).** A plain mass deposit $\sum_j m_j W_{jk}/V$ into a new `rho_dust` (with the additive exchange of item 1), added at the three reads of §2.2; the **g** field, its exchange, and the gather in `ExplicitPush`; skip the dormant-stage solve. The FFT solver is extended too so it remains the oracle (decided).

5. **Particle restart (4d).** `outputs/restart.cpp` ignores the particle arrays; long cluster runs are restart chains.

6. **Diagnostics (4d).** `dust_d` is an NGP binning while the physics deposit is TSC; `pvtk` omits stopping time and mass; the SC14 history's gravitational stress becomes a *total*-gravity stress

once dust contributes to Φ and needs its own dust columns; a Roche-density / Hill-radius clump finder does not exist.

7. **Vertical boundary policy (4c/4d).** Particles reaching the open (diode) z faces and deposits landing in their ghost zones: the Athena-C reference silently discards both; decide explicitly and count the mass.

8. **Refinement (Phase 5).** Dust is fatal on `multilevel` in three places, and upstream particles have no remesh handling at all (`PGID` goes stale after the first regrid; nothing in `mesh_refinement.cpp` touches particles). Needs the deposit exchange across levels and a particle redistribution hook next to `ShearingBox::ReinitAfterMeshUpdate`.

9. **Upstream defect.** `Particles::dtnew` is never assigned on the cosmic-ray path yet is read by `Mesh::NewTimeStep`; the dust module sets it, plain particle runs do not.

7 Input-file interface

Required: `<time>/integrator = imex2+` for `coupling = imex`, `rk2` for `pc2` or `hybrid`; `<particles>/particle_type = dust`, `pusher = imex_dust`, `ppc` (particles per cell, an integer multiple of `nspecies` for lattice placement); `<mesh>/nghost >= 2`; periodic boundaries, or shear-periodic x_1 in 3D; `<hydro>/eos = isothermal` whenever `back_reaction = true`. Shearing-box runs read `<shearing_box>/qshear`, `omega0`, `stratified`.

`<dust>` parameter	default	meaning
`coupling`	imex	`imex` (needs `imex2+`) / `hybrid` / `pc2` (both need `rk2`); see §4.5
`shear_remap`	plm	`dc` / `plm` / `ppmx`: y-remap order of the shear-periodic deposit fold (§5)
`shear_fold`	true	`false`: diagnostic, skip the fold
`hybrid_force_mode`	auto	`auto` / `pc2` / `split_be` (`hybrid` only)
`hybrid_enter_zeta`, `hybrid_enter_chi`	0.5, 0.5	split-BE entry thresholds on ζ, χ
`hybrid_exit_zeta`, `hybrid_exit_chi`	0.25, 0.25	PC2 re-entry thresholds
`nspecies`	1	number of species; `taus_1..N` (positive) required
`dust_to_gas`	0.01	total dust-to-gas ratio for the default mass initializer
`back_reaction`	true	deposit the drag reaction onto the gas
`deposit`	tsc	`ngp` / `cic` / `tsc` assignment and gather kernel
`dt_cfl`	0.5	particle transport CFL factor
`gamma_switch`	false	Krapp et al. eq. 18: $\gamma \to 1/2$ when $\Delta t > \max t_s$
`stopping_time_mode`	species_fixed	or `particle_static`, `dynamic`
`drag_solver`	local	`local` / `dc1` / `dc2` / `pcg` / `adaptive` / `applya` (coupled Schur-complement solvers; `local` is the BLKP19 closed form and the validated baseline)
`drag_rtol`, `drag_atol`, `drag_iter_max`	10^{-11}, 10^{-14}, 200	PCG tolerances
`drag_adaptive_*`	—	error target of the adaptive solver
`drag_diagnostic_interval`	0	`DUST_SOLVER_DIAG` print cadence

Problem-generator parameters (`rho0`, `etavk`, `eps_s`, placement, seeds, eigenmode amplitudes; `dust_nsh`'s `mass_mod_amp`, `mass_mod_phase`, `check_equilibrium`; `dust_deposit_shear`'s `time0`,

`remap`, `fold`) are specific to the five built-in generators `dust_damping`, `dust_nsh`, `dusty_wave`, `streaming_linear`, `dust_deposit_shear`; the files under `inputs/dust/` and `../athenak_dust_tests/inputs/` are the reference configurations. Outputs: `file_type = pvtk` with `variable = prtcl_all` (positions, gid, tag, species, velocities); `variable = dust_d` on any mesh output (NGP mass density, needs a `<dust>` block); `<time>/dtmax` caps the step for fixed-step convergence tests.